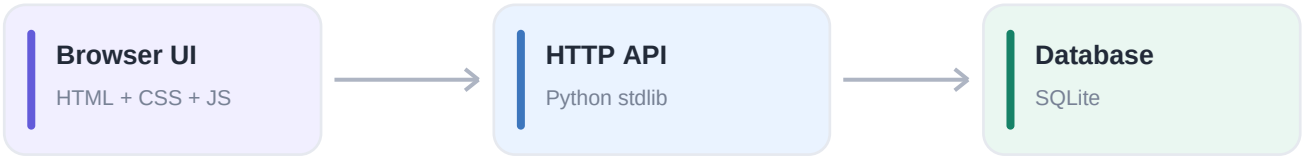
A FULL-STACK LEARNING PROJECT

Shelfwise

A small-shop inventory and order manager built to keep stock accurate and make daily shop work easier.



How the parts work together

THE PROBLEM	THE CORE RULE	THE STACK
A shop needs a reliable view of products, stock, and sales orders.	An order deducts stock as one transaction, or it is rejected without partial changes.	Python standard library, SQLite, and a responsive browser UI.

What this guide covers

The product flow, folder structure, data model, stock-safety transaction, API endpoints, local setup, and a concise interview walkthrough.

Project status: local starter implementation. GitHub publishing is intentionally left for later.

01 / PRODUCT TOUR

What Shelfwise does

Shelfwise gives a small shop one place to check inventory and record orders. The screens are deliberately simple so the business rules are easy to see.

SCREEN	WHAT YOU CAN DO	WHY IT MATTERS
Overview	See active sales, order count, product count, low-stock items, and recent orders.	A shop owner sees the work that needs attention first.
Inventory	Search products, add an item, and update its on-hand stock.	Stock counts and reorder points are visible together.
Orders	Create a multi-item order or cancel a placed order.	Order creation deducts stock; cancellation returns it.

The main user flow

1. Find an item	2. Place an order	3. Stock updates	4. Review the result
Search the catalog and check its available units.	Choose a customer and one or more products.	Stock is deducted only if every requested quantity is available.	The dashboard and order history refresh from the API.

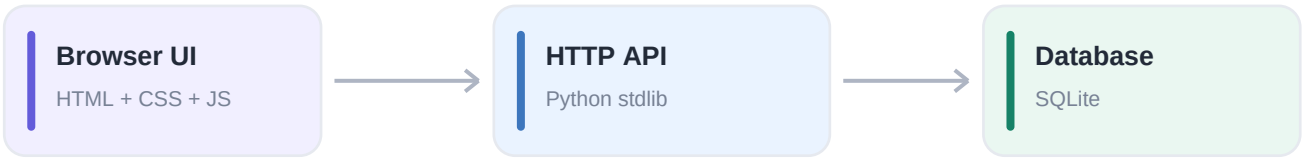
> **Try this after starting the app**
>
> Create an order for a product with enough stock. Then place another order for more than the remaining amount. The second request should be rejected, and inventory should remain unchanged.

The starter data uses sample shop products. You can delete the local database file to reset the demo; the next run seeds the sample catalog again.

02 / ARCHITECTURE

How the code is organized

The browser renders the screens and calls a small JSON API. The API validates requests, applies business rules, and persists the result in SQLite.



PATH	RESPONSIBILITY
backend/server.py	HTTP routes, request validation, database schema, sample data, order transactions, and cancellation logic.
frontend/index.html	The dashboard, inventory and order screens, and accessible product/order dialogs.
frontend/styles.css	Visual styles and responsive behavior for laptop and mobile widths.
frontend/app.js	Fetch calls, form events, view navigation, loading data, and updating the UI.
data/shelfwise.db	Local SQLite file created when the server starts. It is ignored by Git.

A typical request

The browser sends JSON with fetch(). The Python handler routes the request, checks the fields, and uses SQL statements to read or change data. The server sends JSON back; the browser then updates the visible page. Both ends use JSON, so the browser does not need direct database access.

> **Why the UI and server are separate**
>
> The same API rules apply whether an order comes from this screen, a future mobile app, or another client. Keeping the rule on the server prevents a browser-side check from being bypassed.

03 / DATA AND STOCK SAFETY

The important backend decision

Inventory is shared data. If two requests try to sell the last unit at nearly the same time, a read-then-write sequence without a transaction could oversell it.

TABLE	KEY FIELDS	RELATIONSHIP
products	id, name, sku, price_cents, stock, reorder_level	One product can appear in many order_items rows.
orders	id, customer, status, total_cents, created_at	One order owns one or more order_items rows.
order_items	order_id, product_id, product_name, quantity, unit_price_cents	Connects products and orders; stores the name and price at order time.

Order creation, in sequence

01	Open a write transaction with BEGIN IMMEDIATE so another writer cannot change stock during this order.
02	Read each requested product and confirm it exists and has enough stock.
03	Create the order, conditionally deduct stock, and add each order line.
04	Commit only when every step succeeds. If any step fails, rollback removes all changes.

Why store price and name on the order item?

The product catalog can change later. An order should still show the item name and price that applied when it was placed, so order_items stores a snapshot.

Money is stored as whole cents, such as 1899 for $18.99. Integer arithmetic avoids floating-point rounding surprises when calculating totals.

04 / API AND SETUP

How to run and explore it

Shelfwise has no third-party Python packages. Python starts the web server and serves the frontend files from the same local address.

Start the app

```
python backend/server.py
```

Run the command from the project folder. Open **http://127.0.0.1:8000**. Stop the server with Ctrl+C. Requires Python 3.10 or newer.

API reference

METHOD + PATH	WHAT IT DOES	COMMON RESPONSE
GET /api/dashboard	Returns summary metrics, low-stock products, and recent orders.	200 with dashboard JSON
GET /api/products?search=...	Lists products; optional text search checks product name, SKU, and category.	200 with product array
POST /api/products	Creates a catalog item. Price is supplied as cents.	201, or 409 for duplicate SKU
PATCH /api/products/{id}	Updates supported fields, including the on-hand stock value.	200, or 404 if missing
GET /api/orders	Lists orders with saved item snapshots.	200 with order array
POST /api/orders	Places a multi-product order and deducts stock transactionally.	201, or 409 for insufficient stock
POST /api/orders/{id}/cancel	Cancels a placed order once and returns units to stock.	200, or 409 if already cancelled

HTTP status codes

400 means invalid input; 404 means the requested record does not exist; 409 means a business rule prevents the change, such as a duplicate SKU or insufficient stock.

05 / INTERVIEW WALKTHROUGH

How to explain the project

A concise story helps an interviewer understand the problem before looking at individual files.

30-second project summary

I built Shelfwise, a small-shop inventory and order manager. It has a responsive browser UI, a Python JSON API, and SQLite persistence. The core engineering problem is keeping stock correct: each order checks availability and applies every stock deduction in one transaction, so rejected orders do not leave partial changes.

A practical demo sequence

SHOW	SAY
Overview	The summary is calculated from saved orders and current inventory, not hard-coded labels.
Inventory search	A product has a SKU, price, on-hand stock, and reorder point. Search is handled by the API.
Create order	The order may have several lines. The server validates all requested quantities before it commits.
Insufficient stock	A conflict response leaves the order list and stock unchanged. That rule belongs on the server.
Cancel order	A placed order can be cancelled once; the same transaction restores the units.

Good follow-up topics

- Why SQLite is enough for a local demo, and what would change for a multi-user hosted shop.
- Why cents are stored as integers instead of floating-point currency.
- How the transaction protects stock when requests overlap.
- What you would add next: authentication, audit history, automated tests, and backups.

Current limitations

This starter has no sign-in, roles, online deployment, automated test suite, or backup process. Describe those as future work, not as features that already exist.

Keep GitHub for later

The project is saved locally in its own folder. When you are ready to publish it, you can review the files, initialize Git, create a new repository under your account, and push the branch. Nothing in this setup has published it to GitHub.